

Supporting Information

Representing Polymers as Periodic Graphs with Learned Descriptors for Accurate Polymer Property Predictions

Evan R. Antoniuk^{1*}, Peggy Li², Bhavya Kailkhura³, and Anna M. Hiszpanski^{1**}

¹Materials Science Division, Physical and Life Sciences Directorate, Lawrence Livermore National Laboratory, Livermore, CA, USA

²Global Security Computing Applications Division, Computing Directorate, Lawrence Livermore National Laboratory, Livermore, CA, USA

³Machine Intelligence Group/Center for Applied Scientific Computing, Computing Directorate, Lawrence Livermore National Laboratory, Livermore, CA, USA

Co-Corresponding authors: [*antoniuk1@llnl.gov](mailto:antoniuk1@llnl.gov), [**hiszpanski2@llnl.gov](mailto:hiszpanski2@llnl.gov)

Supporting Information

Supplementary Note 1: Featurization of Monomers

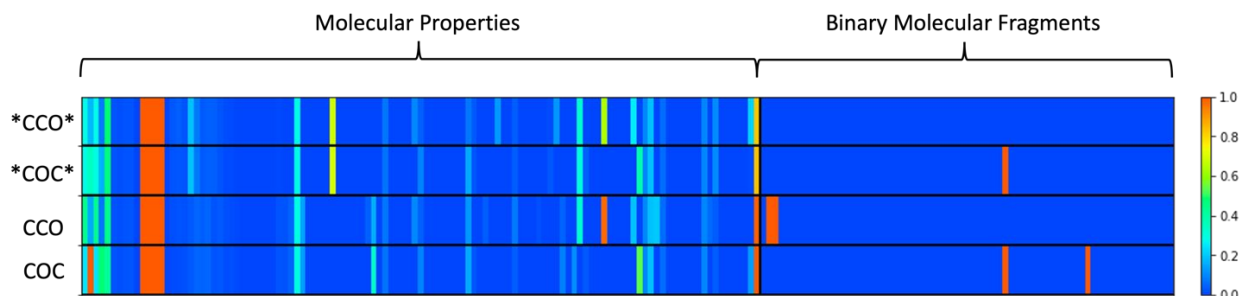


Figure S1. Visualization of molecular descriptors generated in RDKit for 4 representative SMILES strings for polyethylene glycol, $(-\text{CH}_2-\text{CH}_2-\text{O})_n$. The molecular property features are normalized by the maximum value that they have in the full dataset of T_g values.

To illustrate the inconsistencies that arise when using preexisting molecular descriptors to represent polymers, Fig. S1 provides a visualize of the molecular descriptors generated from RDKit for 4 representative SMILES strings. According to the syntax of the SMILES strings, the '*' character is used to indicate the connection between repeating units of a polymer. The SMILES notation also does not explicitly indicate hydrogen atoms, but instead assumes that all atoms have the number of hydrogen atoms required for full valency. The '*CCO*' SMILES string therefore represents a $(-\text{CH}_2-\text{CH}_2-\text{O})_n$ polymer and the '*COC*' SMILES string represents the same polymer, $(-\text{CH}_2-\text{O}-\text{CH}_2)_n$, that has been trivially translated along the polymer backbone. Even though '*CCO*' and '*COC*' represent the same polymer, Fig. S1 illustrates the different feature vectors that are generated from these SMILES strings. An understanding of why this difference arises can be obtained by looking at the molecular fragment features that are generated for these two SMILES string. The only difference in the molecular fragment features is the '*fr_ether*' feature, which is a binary feature that indicates the presence of an ether functional group. As shown in Fig. S1, the '*CCO*' has *fr_ether* = 0 (indicating the absence of an ether), whereas '*COC*' has *fr_ether* = 1 (indicating the presence of an ether). Evidently, the inconsistent featurization between these two chemically identical SMILES strings is due to a

lack of periodicity. Since the '*CCO*' SMILES string does not treat the terminal oxygen atom as being bonded to two neighboring carbon atoms, using the '*CCO*' SMILES string results in an incorrect featurization of $fr_{ether} = 0$, when this polymer does in fact contain an ether.

The impact of including the '*' character in the SMILES representation is illustrated with feature vectors of the 'CCO' and 'COC' SMILES strings, which are the monomer subunits of '*CCO*' and '*COC*', respectively. Interestingly, all four of these inputs yield four different feature vectors. Although these inconsistencies could be accounted for by designating a standardized SMILES string to use as an input for a given polymer, it is unlikely that such a standardized system would always capture the correct periodicity of the polymer. Instead, the use of the *periodic polymer graph* representation as described in the main text ensures that the periodicity of the polymer is maintained and always generates a consistent representation.

Supplementary Note 2: Explanation of Features in Fig. 4

5 Most Correlated Molecular Descriptors

MPNN Feature #1	fr_NH1	NumHDonors	NHOHCount	Fr_aniline	VSA_EState2
MPNN Feature #2	RingCount	NumAromatic Rings	BertzCT	PE0E_VSA14	Fr_benzene
MPNN Feature #3	RingCount	NumAromatic Rings	Fr_benzene	NumAromatic Carbocycles	VSA_EState6
MPNN Feature #4	RingCount	NumAromatic Rings	SlogP_VSA8	Fr_bicyclic	BertzCT
MPNN Feature #5	FractionCSP3	NumAromatic Rings	SMR_VSA7	Fr_benzene	NumAromatic Carbocycles

Polar Group
 Number of Rings
 Other

Figure S2. Same as Fig. 4b, but molecular features are named according to the RDKit documentation.

Here, we list the original names of the features (as described in the RDKit documentation). Each feature in the main text is also categorized into capturing 1) the presence of polar groups, 2) the

number of rings, and 3) other. For the features that do not explicitly represent these groups, we also provide a simple justification for classifying each feature into each group.

fr_NH1: The presence of a NH1 group in the molecule. Polar Groups.

NumHDonors: Number of hydrogen bond donors in the molecule. Polar Groups.

NHOHCount: The total number of NH or OH functional groups in the molecule. Polar Groups.

fr_aniline: The presence of an aniline functional group in the molecule. Since aniline contains both a polar group (NH) as well as a ring, we classify this feature as capturing both 1) presence of polar groups and 2) the number of rings.

VSA_EState2: The electrotopological state of an atom, summed for all atoms in the molecule for a van der Waals surface area between 4.78-5.00. Other.

RingCount: Number of rings in the molecule. Number of rings.

NumAromaticRings: Number of aromatic rings in the molecule. Number of rings.

BertzCT: A topological descriptor that describes the 'complexity' of a molecule. Other.

PEOE_VSA14: Approximate descriptor for describing a molecules surface area. Other.

fr_benzene: The presence of a benzene ring in the molecule. Number of rings.

NumAromaticCarbocycles: Number of aromatic carbocycles in the molecule. Number of rings.

VSA_EState6: Same as VSA_EState2, but calculated for a van der Waals surface area between 6.00-6.07. Other.

SlogP_VSA8: The log P value of a molecule is a partition coefficient that describes its lipophilicity. This descriptor describes the number of atoms in the surface area of molecule that have log P values between 0.25 and 0.30. We include this feature in Group 1 since the lipophilicity of a molecule is heavily impacted by the polarity of a molecule.

fr_bicyclic: Presence of bicyclic substructures in the molecule. Number of rings.

FractionCSP3: Fraction of carbons in the molecule that are SP3 hybridized. Other.

SMR_VSA7: Molar refractivity (MR) describes the polarizability of a molecule. Similar to the SlogP_VSA8 feature, this descriptor describes the number of atoms in the surface area of a molecule that have estimated molar refractivity values between 3.05-3.63. Other.

Table S1. Performance (R^2) of *periodic polymer graph*, trimer graph, dimer graph, monomer graph representations trained with the MPNN architecture as described in the Methods section. The ‘1 repeat’ and ‘3 repeat’ polymer graph models refer to polymer graphs that are generated from graphs of monomers and trimers, respectively. Also shown is the performance of molecular descriptors with a random forest (RF) model. The best performing model (highest value) is highlighted in bold.

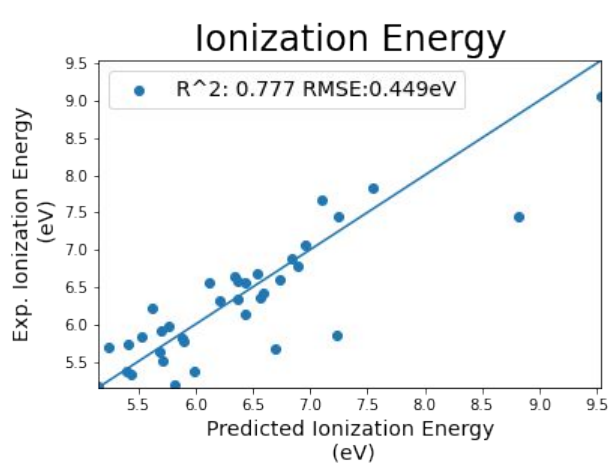
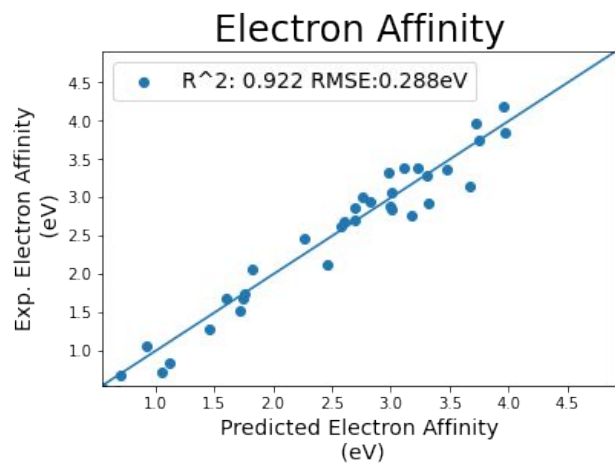
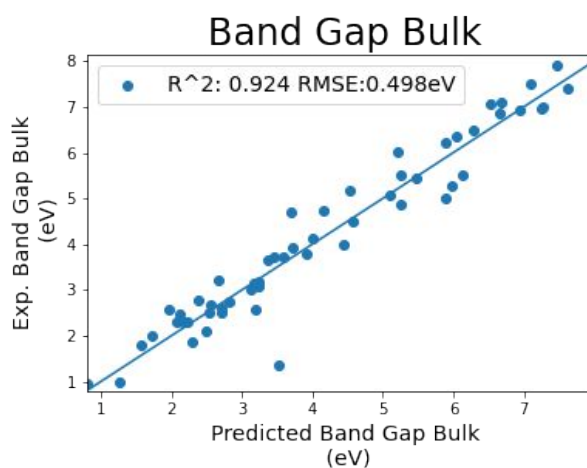
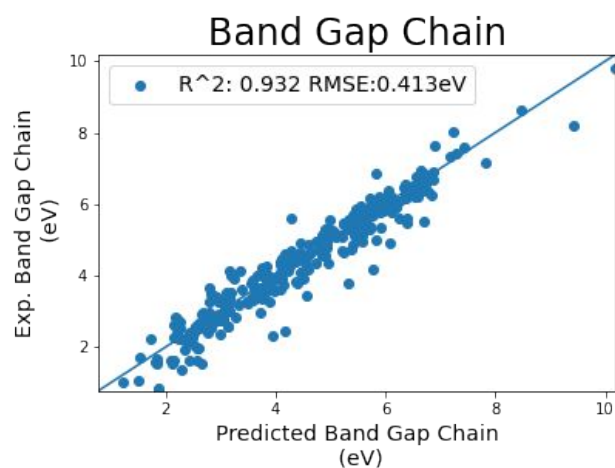
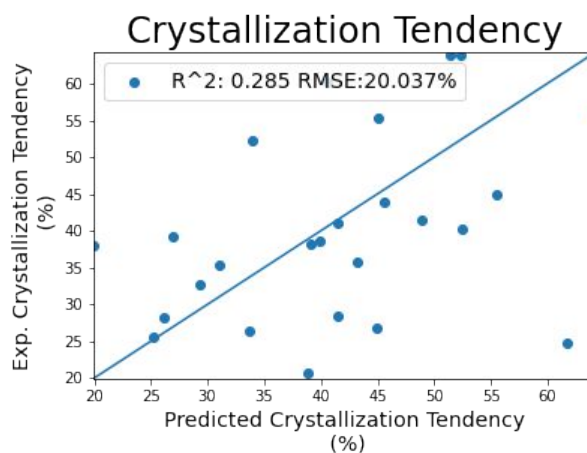
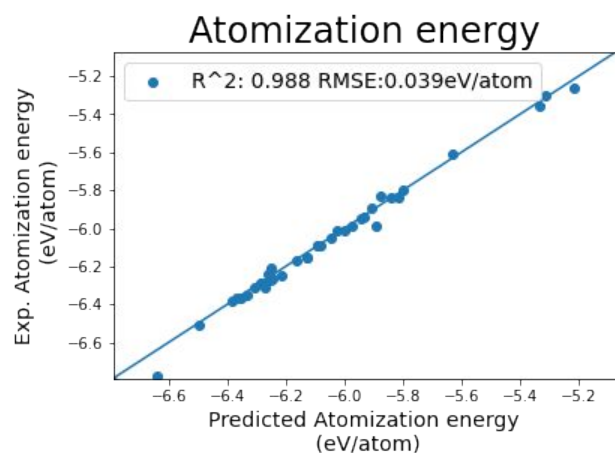
Prop.	Unit	# Data Point	RF/ Molecular Descriptor	MPNN/ Monomer Graph	MPNN/ Dimer Graph	MPNN/ Trimer Graph	MPNN/ Periodic Polymer Graph (1 repeat)	MPNN/ Periodic Polymer Graph (3 repeat)
Atom. Energy	eV/ atom	390	0.917	0.977	0.983	0.984	0.988	0.990
Crystal. Tendency	%	432	0.442	0.238	0.249	0.267	0.285	0.201

Band Gap Chain	eV	3380	0.870	0.887	0.914	0.916	0.932	0.919
Band Gap Bulk	eV	561	0.882	0.882	0.901	0.904	0.924	0.918
Electron Affinity	eV	368	0.832	0.904	0.918	0.920	0.922	0.923
Ionization Energy	eV	370	0.771	0.788	0.811	0.803	0.777	0.795
Refractive Index	-	382	0.815	0.760	0.781	0.780	0.806	0.813
Dielectric Constant	-	382	0.731	0.664	0.677	0.684	0.743	0.672
Glass Transition Temp.	°C	7235	0.874	0.872	0.896	0.899	0.887	0.904
Density	g/cm ₃	1719	0.785	0.787	0.783	0.787	0.796	0.790
Mean R ²	-	-	0.790	0.776	0.791	0.794	0.806	0.793

Table S2. Performance (RMSE) of *periodic polymer graph*, trimer graph, dimer graph, monomer graph representations trained with the MPNN architecture as described in the Methods section. The ‘1 repeat’ and ‘3 repeat’ polymer graph models refer to polymer graphs that are generated from graphs of monomers and trimers, respectively. Also shown is the performance of molecular descriptors with a random forest (RF) model. The mean normalized RMSE is obtained by normalizing the RMSE of a model against the RMSE of the RF model and then averaging across all 10 properties. The best performing model (lowest value) is highlighted in bold.

Prop.	Unit	# Data Point	RF/ Molecular Descriptor	MPNN/ Monomer Graph	MPNN/ Dimer Graph	MPNN/ Trimer Graph	MPNN/ Periodic Polymer Graph (1 repeat)	MPNN/ Periodic Polymer Graph (3 repeat)
Atom. Energy	eV/atom	390	0.097	0.055	0.046	0.045	0.039	0.038
Crystal. Tendency	%	432	18.12	19.52	20.53	20.27	20.04	20.18
Band Gap Chain	eV	3380	0.591	0.509	0.446	0.442	0.413	0.449
Band Gap Bulk	eV	561	0.690	0.614	0.577	0.569	0.498	0.528
Electron Affinity	eV	368	0.444	0.326	0.302	0.297	0.288	0.289
Ionization Energy	eV	370	0.475	0.463	0.436	0.442	0.449	0.385
Refractive Index	-	382	0.108	0.109	0.104	0.104	0.090	0.090
Dielectric Constant	-	382	0.609	0.635	0.621	0.614	0.526	0.535

Glass Transition Temp.	°C	7235	40.70	39.83	36.39	35.84	37.35	34.91
Density	g/cm ³	1719	0.094	0.087	0.087	0.087	0.085	0.086
Mean Normalized RMSE	-	-	1.000	0.906	0.860	0.853	0.804	0.798



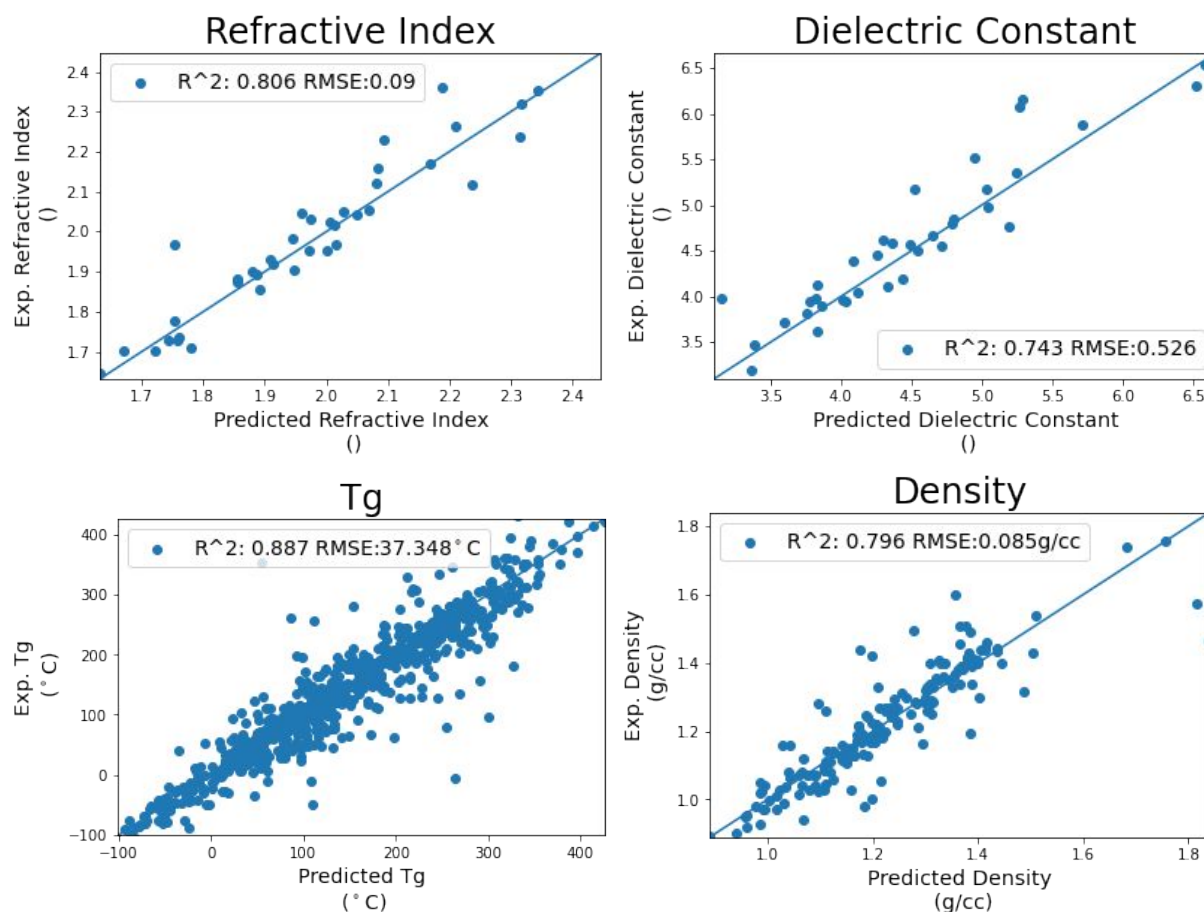


Figure S3. Parity plots for the MPNN/ Periodic Polymer Graph (1 repeat) model and the dataset values.

Supplementary Note 3: Python Code for Utilizing *Periodic Polymer Graphs* in Chemprop

Generating *periodic polymer graphs* can be readily implemented through a slight modification of the Chemprop source code. For the case of generating polymer graphs with three repeat units, given an array of polymer SMILES strings, we generate polymer SMILES strings with three repeat units with the function contained in the file *gen_oligomer_smiles.py*. For example,

running `gen_oligomer_smiles` on the polymer SMILES string `*CCO*`, will yield a polymer SMILES string with three repeat units, `*CCOCCOCCO*`.

This code is used to create a two-column .csv file that contains polymer SMILES strings in the first column and the associated properties in the second column. Then, we use the modified Chemprop *featurization.py* file (found in `chemprop/features/featurization.py`) to contain the additions noted below in bold.

```
...
class MolGraph:
    """
    A :class:`MolGraph` represents the graph structure and featurization of a single molecule.

    A MolGraph computes the following attributes:

    * :code:`n_atoms`: The number of atoms in the molecule.
    * :code:`n_bonds`: The number of bonds in the molecule.
    * :code:`f_atoms`: A mapping from an atom index to a list of atom features.
    * :code:`f_bonds`: A mapping from a bond index to a list of bond features.
    * :code:`a2b`: A mapping from an atom index to a list of incoming bond indices.
    * :code:`b2a`: A mapping from a bond index to the index of the atom the bond originates from.
    * :code:`b2revb`: A mapping from a bond index to the index of the reverse bond.
    * :code:`overwrite_default_atom_features`: A boolean to overwrite default atom descriptors.
    * :code:`overwrite_default_bond_features`: A boolean to overwrite default bond descriptors.
    """

    def __init__(self, mol: Union[str, Chem.Mol, Tuple[Chem.Mol, Chem.Mol]],
                 atom_features_extra: np.ndarray = None,
                 bond_features_extra: np.ndarray = None,
                 overwrite_default_atom_features: bool = False,
                 overwrite_default_bond_features: bool = False):
        """
        :param mol: A SMILES or an RDKit molecule.
        :param atom_features_extra: A list of 2D numpy array containing additional atom features to featurize the molecule
        :param bond_features_extra: A list of 2D numpy array containing additional bond features to featurize the molecule
        :param overwrite_default_atom_features: Boolean to overwrite default atom features by atom_features instead of concatenating
        :param overwrite_default_bond_features: Boolean to overwrite default bond features by bond_features instead of concatenating
        """

        self.is_reaction = is_reaction()
        self.is_explicit_h = is_explicit_h()
        self.reaction_mode = reaction_mode()

        # Convert SMILES to RDKit molecule if necessary
        if type(mol) == str:
            if self.is_reaction:
                mol = (make_mol(mol.split(">")[0], self.is_explicit_h), make_mol(mol.split(">")[-1], self.is_explicit_h))
            else:
                mol = make_mol(mol, self.is_explicit_h)
```

```

mol=Chem.AddHs(mol)
output_mol_array=Chem.MolToMolBlock(mol)
output_mol_array=output_mol_array.split("\n")
n_atoms_mol=mol.GetNumAtoms()
nearest_neighbor=[]
for atom in mol.GetAtoms():
    for neighbor in atom.GetNeighbors():
        if(neighbor.GetAtomicNum() ==0):
            nearest_neighbor.append(atom.GetIdx())
mol_coords=np.array([i.split()[0:3]for i in output_mol_array[4:4+n_atoms_mol]], dtype=float)
self.n_atoms = 0 # number of atoms
self.n_bonds = 0 # number of bonds
self.f_atoms = [] # mapping from atom index to atom features
self.f_bonds = [] # mapping from bond index to concat(in_atom, bond) features
self.a2b = [] # mapping from atom index to incoming bond indices
self.b2a = [] # mapping from bond index to the index of the atom the bond is coming from
self.b2revb = [] # mapping from bond index to the index of the reverse bond
self.overwrite_default_atom_features = overwrite_default_atom_features
self.overwrite_default_bond_features = overwrite_default_bond_features

if not self.is_reaction:
    # Get atom features
    self.f_atoms = [atom_features(atom) for atom in mol.GetAtoms()]
    if atom_features_extra is not None:
        if overwrite_default_atom_features:
            self.f_atoms = [descs.tolist() for desc in atom_features_extra]
        else:
            self.f_atoms = [f_atoms + desc.tolist() for f_atoms, desc in zip(self.f_atoms, atom_features_extra)]

    self.n_atoms = len(self.f_atoms)
    if atom_features_extra is not None and len(atom_features_extra) != self.n_atoms:
        raise ValueError(f'The number of atoms in {Chem.MolToSmiles(mol)} is different from the length of '
            f'the extra atom features')

    # Initialize atom to bond mapping for each atom
    for _ in range(self.n_atoms):
        self.a2b.append([])
    # Get bond features
    for a1 in range(self.n_atoms):
        for a2 in range(a1 + 1, self.n_atoms):
            bond = mol.GetBondBetweenAtoms(a1, a2)
            if bond is None:
                if(a1 in nearest_neighbor and a2 in nearest_neighbor):
                    for neighbor in mol.GetAtomWithIdx(a1).GetNeighbors():
                        if(neighbor.GetAtomicNum() ==0):

                            bond_type=mol.GetBondBetweenAtoms(a1,neighbor.GetIdx())
                            f_bond=bond_features(bond_type)
            else:

```

```
        continue
```

```
...
```

The atom feature vector is also slightly modified to prevent the '*' characters in the SMILES string from being featurized as an atom.

```
...
```

```
def atom_features(atom: Chem.rdchem.Atom, functional_groups: List[int] = None) -> List[Union[bool, int, float]]:
```

```
    """
```

```
    Builds a feature vector for an atom.
```

```
    :param atom: An RDKit atom.
```

```
    :param functional_groups: A k-hot vector indicating the functional groups the atom belongs to.
```

```
    :return: A list containing the atom features.
```

```
    """
```

```
    if atom.GetAtomicNum() == 0 or atom is None:
```

```
        features = [0] * PARAMS.ATOM_FDIM
```

```
    else:
```

```
        features = onek_encoding_unk(atom.GetAtomicNum() - 1, PARAMS.ATOM_FEATURES['atomic_num']) + \
```

```
            onek_encoding_unk(atom.GetTotalDegree(), PARAMS.ATOM_FEATURES['degree']) + \
```

```
            onek_encoding_unk(atom.GetFormalCharge(), PARAMS.ATOM_FEATURES['formal_charge']) + \
```

```
            onek_encoding_unk(int(atom.GetChiralTag()), PARAMS.ATOM_FEATURES['chiral_tag']) + \
```

```
            onek_encoding_unk(int(atom.GetTotalNumHs()), PARAMS.ATOM_FEATURES['num_Hs']) + \
```

```
            onek_encoding_unk(int(atom.GetHybridization()), PARAMS.ATOM_FEATURES['hybridization']) + \
```

```
            [1 if atom.GetIsAromatic() else 0] + \
```

```
            [atom.GetMass() * 0.01] # scaled to about the same range as other features
```

```
    if functional_groups is not None:
```

```
        features += functional_groups
```

```
    return features
```

```
...
```

Following these modifications, the Chemprop program can be trained as usual, for example, with the command: `chemprop_train --data_path polymers.csv --dataset_type regression --save_dir checkpoints`, where `polymers.csv` is the `.csv` file that contains the polymer SMILES strings and their associated properties.